AI SaaS Video Ad Technical & Creative Skills Guide

This document distills technical specs (from ffprobe) and narrative patterns (from transcripts) of the pr

1. Encoding and Delivery Baseline

- Resolution: All major ads are exported at 1920x1080 (16:9) with H.264 video and AAC stereo audio.
- Frame rate: They target 25 fps progressive, which is a good baseline for web ads and product videos
- Bitrate: Overall file bitrates range roughly from ~1.2 Mbps to ~2.1 Mbps, with audio around 128 kbps
- Container: MP4 (QuickTime / MOV compatible brands isom/iso2/avc1/mp41).

These values give you safe, high-quality export targets from Remotion (or any renderer) that balance

2. Audio Structure and Voiceover Patterns

From the transcripts:

- Lovio launch: Starts with a problem-first hook ("You don't start with a product. You start with a proble
- Lovio brand: Asks a hypothetical question ("What if building software felt as effortless as having an i
- LangEase: Uses a direct instructional tone ("I'll show you how in under 30 seconds"), then walks thro
- Teamble: Relies heavily on on-screen text and UI to show an internal workflow, with sparse or no sp
- Bud/AI Human Emulator and workflow clips: Use rapid task commands ("Take these user interviews

Core audio skills:

- Write a 45–90 second script with a single narrator, cut into 1–2 sentence beats that each map to a vi
- Use a problem → solution → outcome → CTA structure.
- Mix narration with short bursts of emphasized phrases on screen (e.g., "Instantly", "9x better feedbac
- Keep actual spoken vocabulary simple, focusing on outcomes ("go global", "turn ideas into products"

When implementing with Remotion, you want a script object model (an array of beats with {startFrame

3. Visual Language and Motion Design Patterns

From the Teamble and Lovio videos:

- Camera is essentially screen-space: you see app UIs, cards, and overlays sliding, scaling, fading.
- Motions are fast but eased: 8–16 frames for small moves, 16–24 for bigger transitions.
- Common patterns:
  - Slide-in UI panels from sides or bottom.
  - Zoom-in on key components (cards, charts, scores).
  - Highlight areas with subtle glow, outline, or focus blur elsewhere.
  - Number/score increments (e.g., feedback score from 9/100 to 92/100).
  - Text appearing word-by-word or line-by-line.
- Backgrounds are clean, often light gray or gradient, with strong brand accent colors.

Skill checklist for visuals:

- Component library for UI primitives (cards, buttons, chips, charts) that can be re-skinned per brand.
- Timeline-driven animations with spring or ease curves (e.g., using Remotion's spring and Easing AP
- A utility to define "scenes" and transitions between scenes (cut, crossfade, slide, push).

4. Narrative Structure Templates

You can standardize 3 main templates that cover most clips:

1. **Problem → Solution → Outcome (Lovio-style)**
   - Hook: One sentence about the problem.
   - Reveal: Introduce the product with a simple phrase ("Meet X.").
   - Mechanism: 2–3 beats explaining how it works in simple steps.
   - Outcome: Language about speed, scale, or quality.
   - CTA: Invite to start, try, or sign up.

2. **Mini-tutorial (LangEase-style)**
   - Time promise: "I'll show you in under 30 seconds".
   - Step 1: Input ("Drop in any document, video, or audio").
   - Step 2: AI transformation ("AI translates into multiple languages instantly").
   - Step 3: Extra value ("Need subtitles or voiceover? We've got that").
   - Outcome and CTA.

3. **Workflow transformation (Teamble/Bud-style)**
   - Current behavior: show raw text / poor feedback / manual process.
   - AI analysis: scores, labels (Specificity, Tone, Actionability), insights.
   - AI suggestions: re-written text, generated assets, dashboards.
   - Impact: better scores, uncovered trends, automated output.

Define these templates as TypeScript types (e.g., union of script archetypes) and write generators tha

5. Timing and Beat Lengths

Based on durations and transcripts:

- Lovio clips: ~70–75 seconds, around 8–12 distinct beats.
- Teamble: ~98 seconds with many micro-beats (~2–4 seconds each) driven by UI steps.
- LangEase: very short, ~20–30 second promise with 3–5 beats.

Practically, target:

- Hook: 2–3 seconds.
- Each step: 4–6 seconds.
- Major visual transformations: at most every 3–4 seconds to keep attention.

In Remotion, you can define a constant FPS (e.g., 30 fps for simplicity, even though source ads use 2

6. Typography and On-screen Text

Patterns from Teamble and the ad-style videos:

- Use large, bold headings ("Unlock actionable insights", "The People Success Platform").
- Short, stacked phrases split over multiple lines for rhythm ("Give / your team / superpowers").
- Inline labels: scores, tags like "Tone: Very aggressive", categories (Marketing 2%, Sales 3%).
- Onboarding/questions sequences (for Teamble) laid out in dense but legible UI.

Skill components:

- A responsive text block component that can:
  - Animate in from opacity 0 → 1 with y-translation.
  - Support multi-line layout with controlled line breaks.
  - Optionally reveal line-by-line.
- A number ticker component for animating scores and percentages.

7. Sound Design

Observations:

- Many clips rely on a single music bed (corporate, upbeat, minimal vocals) that runs the full duration.
- Voiceover sits clearly on top with no heavy sound effects.
- No abrupt music cuts, just intro/outro fades and subtle ducking behind VO.

Technical skills:

- Pick royalty-free tracks that match BPM and mood.
- Pre-normalize VO to a consistent loudness (e.g., -16 LUFS) and music below that (e.g., -24 to -20 LU
- Use simple fade-in/out and level automation rather than complex SFX.

You can pre-bounce music and VO and just time-align in Remotion, or render separate stems and mix

8. FFmpeg/FFprobe Usage Patterns

For each exported video, useful commands include:

- Structural inspection:
  - `ffprobe -v quiet -print_format json -show_format -show_streams input.mp4`
- Thumbnail extraction at key frames (for QA or docs):
  - `ffmpeg -ss 5 -i input.mp4 -frames:v 1 thumb_005s.png`
- Re-encoding to target spec (if needed):
  - `ffmpeg -i input.mov -c:v libx264 -profile:v high -level 4.0 -pix_fmt yuv420p -r 25 -b:v 2000k -c:a aac

Automatable skill:

- Create a small CLI or script that takes a Remotion-rendered ProRes/H.264 file and normalizes it to th

9. Remotion Implementation Skills

To reproduce this style in Remotion, you need:

1. **Composition design**
   - Single main composition at 1920x1080, FPS 30 (or 25), duration frames derived from script length.
   - Sub-compositions for complex sequences (e.g., Teamble feedback improvement sequence) that yo

2. **Timeline/layout abstraction**
   - A `Scene` abstraction with fields: `id`, `startFrame`, `durationInFrames`, `type`, `props`.
   - Components for each scene type: `ProblemScene`, `SolutionScene`, `WorkflowScene`, `MetricsSc

3. **Data-driven UI rendering**
   - Feed in JSON that describes fake data (scores, names, companies) and generate cards and charts

   - Use consistent spacing, corner radii, and shadows for a system look.

4. **Animation utilities**
   - Helpers around `useCurrentFrame`, `interpolate`, and `spring` for entrance/exit animations.
   - A pattern like: `enterFromLeft`, `enterFromBottom`, `fadeInUp`, `scaleIn` helpers.

5. **Audio synchronization**
   - Map transcript lines to timestamps, then map timestamps to frames.
   - Align scene `startFrame` to the VO timeline rather than hard-coded guesses.

10. Skill File / Playbook Structure

Inside your codebase, consider these skill modules:

- `skills/script-writing.ts`: functions to generate hooks, CTAs, and steps from product metadata.
- `skills/scene-graph.ts`: types and helpers to map script beats into scenes with timing.
- `skills/ui-kit.tsx`: reusable UI components (cards, charts, overlays, pills).
- `skills/animation.ts`: reusable animation helpers.
- `skills/audio.ts`: utilities for VO/music alignment and loudness-aware volume curves.
- `skills/export-pipeline.md`: documentation for ffmpeg commands, target specs, and QA checklist.

Each skill module should be documented with examples and constraints so you can quickly reuse ther